

Universidade Federal de Uberlândia

Aula 4: Controle sem modelo e aproximação de função valor

MC/TD em lote • ϵ -guloso e GLIE • SARSA e Q-learning • VFA • DQN

Prof. Marcelo Keese Albertini
Universidade Federal de Uberlândia
adaptado de Emma Brunskill, CS234 (Stanford)

2026



Plano de hoje

A aula em que o agente deixa de observar e passa a decidir

1. **Avaliação de política em lote** — fechamos a Aula 3 entendendo *o que* MC e TD calculam quando os dados acabam.
2. **Melhoria de política generalizada** — por que precisamos de Q e não de V , e como explorar sem perder a garantia de melhoria monotônica.
3. **Controle tabular sem modelo** — MC com ϵ -guloso, GLIE, SARSA e Q-learning.
4. **Aproximação de função valor** — abandonar a tabela para *generalizar* entre estados.
5. **Deep Q-Learning** — o que foi preciso acrescentar para que a ideia funcionasse em Atari.

Intuição

Fio condutor: os quatro algoritmos de hoje são a mesma linha de código, $estimativa \leftarrow estimativa + \alpha (alvo - estimativa)$, com quatro escolhas diferentes de *alvo*.

Onde estamos

Duas dimensões: temos o modelo? queremos avaliar ou decidir?

	Avaliar π (predição)	Escolher π (controle)
Com modelo	Avaliação de política (Aula 2)	Iteração de política / de valor (Aula 2)
Sem modelo	MC, TD(0) (Aula 3)	MC-controle, SARSA, Q-learning (hoje)

- ▶ Até agora tudo era **tabular**: uma entrada de memória por estado (ou por par estado-ação).
- ▶ A segunda metade da aula quebra essa hipótese: $|S|$ grande demais para caber em memória, quanto mais para ser visitado.

Recapitulação: os três estimadores da Aula 3

Estimar $V^\pi(s)$ executando π , sem conhecer P e R

- ▶ **Monte Carlo** — precisa da trajetória completa τ e usa o retorno observado G_t :

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(G_t(s) - V^\pi(s))$$

- ▶ **TD(0)** — precisa apenas da tupla (s, a, r, s') e *bootstrapa* na própria estimativa:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha \underbrace{(r + \gamma V^\pi(s') - V^\pi(s))}_{\text{alvo TD}}$$

- ▶ **Equivalência de certeza** — usa as tuplas para estimar $\hat{P}$ e $\hat{R}$ por máxima verossimilhança e roda programação dinâmica sobre esse MDP estimado.

Intuição

Os três já apareceram como *procedimentos*. Hoje começamos perguntando algo mais fino: para *qual objeto matemático* cada um converge quando os dados são finitos e reciclados?

SEÇÃO 1

Avaliação de política em lote

01

Do fluxo para o lote

Batch (ou *offline*): o conjunto de dados é fixo

- ▶ Nas versões vistas até aqui, cada tupla (s, a, r, s') é usada **uma vez** e descartada.
- ▶ No cenário em lote temos K episódios armazenados e podemos reprocessá-los quantas vezes quisermos.

Algoritmo — Avaliação de política em lote

- 1: Dado o conjunto fixo de K episódios
- 2: **repita** até convergir
- 3: amostre um episódio $k \sim \{1, \dots, K\}$
- 4: aplique a atualização de MC (ou de TD(0)) a esse episódio
- 5: **fim**

O exemplo A-B

Sutton & Barto (2018), Exemplo 6.4 — dois estados, $\gamma = 1$

Oito episódios observados:

Episódio	Frequência
$A, 0, B, 0$	1 vez
$B, 1$	6 vezes
$B, 0$	1 vez

Cada episódio termina após a última recompensa. Note que A aparece **uma única vez** em toda a base.

Definição

Estimativa a partir de B : em 8 visitas houve recompensa 1 em 6 delas, logo $V(B) = 6/8 = 0,75$ tanto por MC quanto por TD.

Teste sua compreensão

E $V(A)$? Rodando as atualizações sobre esses dados um número infinito de vezes, para que valor cada método converge?

Pergunta para discussão conduzida em aula.

O exemplo A-B: as duas respostas

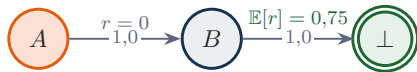
Não é um erro de um dos métodos — eles otimizam critérios diferentes

Resposta

$$V_{MC}(A) = 0 \quad V_{TD}(A) = 0,75$$

Monte Carlo. O único retorno já observado a partir de A foi $G = 0 + 0 = 0$. MC minimiza o erro quadrático em relação aos retornos *observados*, então responde 0.

TD(0). O alvo é $r + \gamma V(B) = 0,75$: TD encadeia a informação obtida em B , mesmo que A apareça uma vez só.



Intuição

Esse é o MDP de **máxima verossimilhança** implícito nos dados. TD(0) em lote converge exatamente para o V^π desse MDP: por isso $V(A) = 0 + 0,75$.

Para onde MC e TD convergem no cenário em lote

Monte Carlo em lote

Converge para os valores que **minimizam o erro quadrático médio** em relação aos retornos observados, $V = \arg \min_V \sum_{k,t} (G_{k,t} - V(s_{k,t}))^2$. No exemplo A-B: $V(A) = 0$.

TD(0) em lote

Converge para o V^π do MDP de máxima verossimilhança estimado dos dados — ou seja, é **idêntico à equivalência de certeza**:

$$\hat{P}(s' | s, a) = \frac{\sum_k \mathbf{1}\{s_k = s, a_k = a, s_{k+1} = s'\}}{N(s, a)}, \quad \hat{R}(s, a) = \frac{\sum_k \mathbf{1}\{s_k = s, a_k = a\} r_k}{N(s, a)}$$

No exemplo A-B: $V(A) = 0,75$.

Qual das duas respostas é “a certa”?

Depende inteiramente de acreditarmos, ou não, na hipótese de Markov

Intuição

Se o ambiente é markoviano, TD está certo: o que se aprende sobre B vale para *qualquer* caminho que leve a B . TD usa a estrutura; MC a ignora.

Atenção

Se o ambiente não é markoviano (estado parcialmente observado, por exemplo), TD propaga uma hipótese falsa. MC, que só olha retornos completos, é mais robusto — ao custo de variância.

MC é correto *sem* supor Markov; TD supõe Markov e, quando ela vale, converge com muito menos dados.

Teste sua compreensão

Com prontuários médicos incompletos, você confiaria mais em MC ou em TD? Por quê?

Cr terios para comparar algoritmos de avalia  o

Custo computacional

- ▶ TD(0): $O(1)$ por transi  o, $O(L)$ por epis  dio.
- ▶ MC: tamb  m $O(L)$, mas s   *depois* do fim do epis  dio.
- ▶ Equival  ncia de certeza: resolve um sistema linear a cada atualiza  o — muito mais caro.

	MC	TD(0)	Eq. cert.
Usa Markov	n��o	sim	sim
Vi��s	n��o	sim	sim
Vari��ncia	alta	baixa	baixa
Epis��dio completo	sim	n��o	n��o
Custo/atualiz.	$O(1)$	$O(1)$	alto
Efici��ncia dados	baixa	m��dia	alta

Efici  ncia de dados

- ▶ Usar a estrutura de Markov (TD, eq. de certeza) extrai mais de cada amostra; MC pode ser prefer  vel quando ela n  o vale.

Fecho da avaliação de política

Estimar o retorno esperado de uma π dada, sem o MDP verdadeiro

- ▶ **Monte Carlo** — com dados *on-policy* ou *off-policy* (amostragem por importância).
- ▶ **Diferença temporal** — TD(0) e suas variantes.
- ▶ **Programação dinâmica com equivalência de certeza** — construir o modelo e resolvê-lo.

Definição — A régua que usaremos o curso inteiro

Supõe Markov? É consistente? Qual o viés e a variância? Quantos dados precisa? Quanto custa por atualização?

Intuição

Avaliação é só metade do problema. A partir de agora queremos *escolher* a política, não apenas medi-la — e aí aparece um obstáculo que não existia até aqui: precisamos de dados sobre ações que a política atual **não** toma.

SEÇÃO 2

Melhoria de política generalizada

02

Iteração de política sem modelo

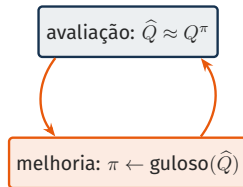
O mesmo esqueleto da Aula 2 — com três buracos

Algoritmo — Iteração de política generalizada

- 1: Inicialize π
- 2: **repita**
- 3: *Avaliação*: estime Q^π
- 4: *Melhoria*: atualize π a partir de Q^π
- 5: **fim**

Atenção

1. Se π é determinística, $Q(s, a)$ com $a \neq \pi(s)$ nunca é observado.
2. A melhoria usa um $\hat{Q}$ estimado, com ruído.
3. Quanto avaliar antes de melhorar?



converge para π^* e Q^*

Por que Q e não V

Uma consequência silenciosa de não ter o modelo

- ▶ Com modelo, a melhoria gulosa se faz a partir de V :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

- ▶ Sem modelo, R e P são desconhecidos — a expressão acima é inútil. Já com Q :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$$

e a melhoria é imediata, sem nenhum conhecimento da dinâmica.

Intuição

Q é exatamente V com a *informação de um passo do modelo já embutida*. É por isso que praticamente todo algoritmo de controle sem modelo aprende Q , e não V .

O problema da exploração

O dilema que separa RL de aprendizado supervisionado

Explorar

- ▶ Não há como aprender sobre uma ação sem experimentá-la.
- ▶ Dados sobre a só existem se π der probabilidade positiva a a .

Uma política **gulosa pura** sobre $\hat{Q}$ pode travar para sempre: basta que a primeira amostra de uma ação boa seja ruim por azar para que ela nunca mais seja escolhida — nem corrigida.

Explotar

- ▶ Cada passo experimentando é um passo não gasto colhendo recompensa conhecida.
- ▶ Em saúde e educação, explorar tem custo real.

Teste sua compreensão

O rover tenta “ir para a direita”, recebe 0 por azar. O que faz uma política gulosa pura daí em diante?

Políticas ϵ -gulosas

A solução mais simples que funciona — e a mais usada na prática

Definição — Política ϵ -gulosa em relação a $Q(s, a)$

Seja $|\mathcal{A}|$ o número de ações e $\epsilon \in [0, 1]$. Então

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{se } a = \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{caso contrário.} \end{cases}$$

- ▶ Toda ação tem probabilidade $\geq \epsilon/|\mathcal{A}| > 0$ — exploração garantida.
- ▶ $\epsilon = 1$: passeio aleatório. $\epsilon = 0$: guloso puro.
- ▶ Alternativa comum: *softmax*, $\pi(a \mid s) \propto e^{Q(s,a)/\tau}$, que explora preferencialmente as ações *quase* boas.

A melhoria monotônica sobrevive à exploração

- ▶ Na Aula 2 provamos que a iteração de política, com P e R conhecidos, melhora monotonicamente — mas a prova supunha que a melhoria produzisse uma política **determinística**.
- ▶ Uma política ϵ -gulosa é estocástica de propósito. A garantia se perde?

Teorema — melhoria monotônica ϵ -gulosa

Seja π_i uma política ϵ -gulosa qualquer e seja π_{i+1} a política ϵ -gulosa em relação a Q^{π_i} . Então

$$V^{\pi_{i+1}}(s) \geq V^{\pi_i}(s) \quad \forall s \in \mathcal{S}.$$

Intuição

Ou seja: é possível explorar *para sempre* e ainda assim nunca piorar. O custo é que a política ótima entre as ϵ -gulosas não é π^* — ela carrega o ϵ de ruído. É esse resíduo que a condição GLIE, adiante, elimina.

Prova da melhoria monotônica ϵ -gulosa

O truque é escrever o máximo como uma média ponderada disfarçada

$$\begin{aligned} Q^{\pi_i}(s, \pi_{i+1}(s)) &= \sum_{a \in \mathcal{A}} \pi_{i+1}(a | s) Q^{\pi_i}(s, a) \\ &= \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q^{\pi_i}(s, a) + (1 - \epsilon) \max_a Q^{\pi_i}(s, a) \\ &\geq \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q^{\pi_i}(s, a) + (1 - \epsilon) \underbrace{\sum_{a \in \mathcal{A}} \frac{\pi_i(a | s) - \epsilon/|\mathcal{A}|}{1 - \epsilon} Q^{\pi_i}(s, a)}_{\text{pesos } \geq 0, \text{ somam } 1} \\ &= \sum_{a \in \mathcal{A}} \pi_i(a | s) Q^{\pi_i}(s, a) = V^{\pi_i}(s). \end{aligned}$$

A desigualdade vale porque o máximo é maior ou igual a *qualquer* combinação convexa. Os pesos são não-negativos precisamente porque π_i é ϵ -gulosa, logo $\pi_i(a | s) \geq \epsilon/|\mathcal{A}|$.

De $Q^{\pi_i}(s, \pi_{i+1}(s)) \geq V^{\pi_i}(s)$ segue $V^{\pi_{i+1}} \geq V^{\pi_i}$ pelo teorema de melhoria de política da Aula 2. ■

SEÇÃO 3

Controle tabular sem modelo

03

Monte Carlo para Q

A avaliação da Aula 3, com o par (s, a) no lugar de s

Algoritmo — Avaliação de política Monte Carlo para Q^π (primeira visita)

- 1: Inicialize $Q(s, a) = 0$, $N(s, a) = 0$ para todo (s, a) ; $k = 1$; política π
- 2: **repita**
- 3: gere o k -ésimo episódio $(s_{k,1}, a_{k,1}, r_{k,1}, \dots, s_{k,T})$ seguindo π
- 4: calcule $G_{k,t} = r_{k,t} + \gamma r_{k,t+1} + \gamma^2 r_{k,t+2} + \dots$ para todo t
- 5: **para** $t = 1, \dots, T$ **faça**
- 6: **se** é a primeira visita a (s_t, a_t) no episódio k **então**
- 7: $N(s_t, a_t) += 1$
- 8: $Q(s_t, a_t) += \frac{1}{N(s_t, a_t)} (G_{k,t} - Q(s_t, a_t))$
- 9: **fim**
- 10: **fim**; $k += 1$
- 11: **fim**

Controle Monte Carlo *on-policy*

Avaliar e melhorar a cada episódio, com ϵ decrescente

Algoritmo — Controle MC *on-policy* com ϵ -guloso

```
1: Inicialize  $Q(s, a) = 0$ ,  $N(s, a) = 0$  para todo  $(s, a)$ ;  $\epsilon = 1$ ;  $k = 1$   
2:  $\pi_k \leftarrow \epsilon\text{-guloso}(Q)$  // política inicial  
3: repita  
4:   gere o episódio  $k$  seguindo  $\pi_k$  e calcule os retornos  $G_{k,t}$   
5:   para  $t = 1, \dots, T$  faça  
6:     se primeira visita a  $(s_t, a_t)$  então  $N \leftarrow N + 1$ ;  $Q(s_t, a_t) \leftarrow \frac{1}{N(s_t, a_t)} (G_{k,t} + Q(s_t, a_t))$   
7:   fim  
8:    $k \leftarrow k + 1$ ;  $\epsilon \leftarrow 1/k$  // exploração decrescente  
9:    $\pi_k \leftarrow \epsilon\text{-guloso}(Q)$  // melhoria de política  
10: fim
```

Exemplo trabalhado: MC-controle no Mars Rover

Sete estados, duas ações, $\gamma = 1$



- ▶ Recompensas: $R(\cdot, a_1) = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ +10]$, $R(\cdot, a_2) = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ +5]$.
- ▶ Início: $Q(s, a) = 0$ para todo par, $\pi(s) = a_1$, $\epsilon = 0,5$.
- ▶ Trajetória amostrada da política ϵ -gulosa:

$$\tau = (s_3, a_1, 0, s_2, a_2, 0, s_3, a_1, 0, s_2, a_2, 0, s_1, a_1, 1, \perp)$$

Teste sua compreensão

Qual é a estimativa MC de primeira visita de Q para cada par (s, a) após essa única trajetória? E qual passa a ser a política gulosa?

Pergunta para discussão conduzida em aula.

Exemplo trabalhado: solução

Com $\gamma = 1$, todo retorno é a soma das recompensas até o fim

Resposta

- ▶ $Q(\cdot, a_1) = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0]$
- ▶ $Q(\cdot, a_2) = [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0]$
- ▶ Nova política gulosa:
 $\pi = [a_1 \ a_2 \ a_1 \ \text{empate} \ \dots]$

Toda primeira visita ocorre antes da recompensa 1 em s_1 : com $\gamma = 1$, cada par visitado recebe retorno 1.

Teste sua compreensão

Com $\epsilon = 1/3$ e $|\mathcal{A}| = 2$, qual a probabilidade de a nova política escolher a_1 em s_1 ?

Exemplo trabalhado: solução

Com $\gamma = 1$, todo retorno é a soma das recompensas até o fim

Resposta

- ▶ $Q(\cdot, a_1) = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0]$
- ▶ $Q(\cdot, a_2) = [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0]$
- ▶ Nova política gulosa:
 $\pi = [a_1 \ a_2 \ a_1 \ \text{empate} \ \dots]$

Toda primeira visita ocorre antes da recompensa 1 em s_1 : com $\gamma = 1$, cada par visitado recebe retorno 1.

Teste sua compreensão

Com $\epsilon = 1/3$ e $|\mathcal{A}| = 2$, qual a probabilidade de a nova política escolher a_1 em s_1 ?

Resposta

$$1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} = 1 - \frac{1}{3} + \frac{1}{6} = \frac{5}{6}.$$

Erro comum: responder $2/3$, esquecendo que a exploração uniforme também pode cair na ação gulosa.

Guloso no limite com exploração infinita (GLIE)

A condição que transforma “nunca piora” em “converge para o ótimo”

Definição — GLIE (*Greedy in the Limit with Infinite Exploration*)

Uma sequência de políticas $\{\pi_i\}$ é GLIE se

1. todo par (s, a) é visitado infinitas vezes, $\lim_{i \rightarrow \infty} N_i(s, a) \rightarrow \infty$;
2. a política de comportamento converge para a gulosa,
 $\lim_{i \rightarrow \infty} \pi_i(a \mid s) \rightarrow \arg \max_a Q(s, a)$ com probabilidade 1.

As duas condições puxam em direções opostas. A **estratégia GLIE mais simples** concilia ambas: ϵ -guloso com $\epsilon_i = 1/i$, pois $\sum_i 1/i$ diverge mas $\epsilon_i \rightarrow 0$.

Teorema — convergência do controle MC com GLIE

O controle Monte Carlo GLIE converge para a função valor de ação ótima: $Q(s, a) \rightarrow Q^*(s, a)$.

Controle com métodos de diferença temporal

Mesma receita, mas agora aprendendo a cada *passo*

Algoritmo — Iteração de política generalizada com TD

- 1: Inicialize π (e Q)
- 2: **repita**
- 3: *Avaliação*: atualize Q por diferença temporal usando dados da ϵ -gulosa
- 4: *Melhoria*: $\pi \leftarrow \epsilon\text{-guloso}(Q)$
- 5: **fim**

Vantagem imediata: não é preciso esperar o episódio terminar — funciona em tarefas **contínuas**. E a pergunta que separa os dois algoritmos de hoje: **qual Q do próximo estado entra no alvo?**

Intuição

SARSA usa o valor da ação que o agente *de fato tomou*. **Q-learning** usa o valor da *melhor* ação disponível, tenha ela sido tomada ou não. Uma diferença de um símbolo, com consequências grandes.

Definição — Aprendizado on-policy

Estima e melhora a política *a partir da experiência gerada por ela mesma*. A política que age é a política que se aprende.

Exemplos: controle MC on-policy, SARSA.

Definição — Aprendizado off-policy

Estima e melhora uma política *alvo* usando dados coletados por uma política de *comportamento* diferente, π_b .

Exemplos: Q-learning, DQN, praticamente todo RL offline moderno.

- ▶ Off-policy é o que permite aprender de **dados de arquivo**: registros médicos, logs de recomendação, demonstrações humanas.
- ▶ Também permite reutilizar experiência antiga — a base do *replay* do DQN — mas é a fonte de boa parte da instabilidade que veremos ao combiná-lo com aproximação de função.

SARSA

O nome vem da quintupla usada na atualização: (s, a, r, s', a')

Definição — Atualização SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(\underbrace{r_t + \gamma Q(s_{t+1}, a_{t+1})}_{\text{alvo: ação realmente tomada}} - Q(s_t, a_t) \right)$$

Algoritmo — SARSA com exploração ϵ -gulosa

- 1: Inicialize $Q(s, a)$; $t = 0$; estado inicial s_0 ; $\pi \leftarrow \epsilon\text{-guloso}(Q)$
- 2: Tome $a_t \sim \pi(s_t)$
- 3: **repita**
- 4: observe (r_t, s_{t+1}) e tome $a_{t+1} \sim \pi(s_{t+1})$
- 5: $Q(s_t, a_t) += \alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$
- 6: $\pi \leftarrow \epsilon\text{-guloso}(Q)$; $t += 1$
- 7: **fim**

Q-learning

Aprender Q^* enquanto se age com outra política

Definição — Atualização Q-learning (Watkins, 1989)

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(\underbrace{r_t + \gamma \max_{a'} Q(s_{t+1}, a')}_{\text{alvo: melhor ação possível}} - Q(s_t, a_t) \right)$$

Algoritmo — Q-learning com exploração ϵ -gulosa

- 1: Inicialize $Q(s, a)$ para todo $s \in \mathcal{S}$, $a \in \mathcal{A}$; $t = 0$; estado inicial s_0
- 2: Defina $\pi_b \leftarrow \epsilon$ -guloso(Q) *// política de comportamento*
- 3: **repita**
- 4: tome $a_t \sim \pi_b(s_t)$ e observe (r_t, s_{t+1})
- 5: $Q(s_t, a_t) += \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$
- 6: $\pi_b \leftarrow \epsilon$ -guloso(Q); $t += 1$
- 7: **fim**

Convergência do Q-learning tabular

Teorema

Para MDPs com $\mathcal{S}$ e $\mathcal{A}$ finitos, o Q-learning converge para o valor de ação ótimo, $Q(s, a) \rightarrow Q^*(s, a)$, desde que **(i)** a sequência de políticas satisfaça a condição GLIE e **(ii)** os passos satisfaçam as condições de Robbins–Monro:

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

Intuição

A primeira soma garante que qualquer erro inicial *pode* ser corrigido (os passos não somem cedo demais); a segunda garante que o ruído das amostras é amortecido. Ex.: $\alpha_t = 1/t$ serve.

A prova apoia-se em aproximação estocástica, na contração do backup de Bellman e em valores limitados. **Nenhuma das três sobrevive intacta à aproximação de função.**

SARSA vs. Q-learning no mesmo passo

Mars Rover com $\alpha = 0,5$, $\gamma = 1$, $Q(s_7, a_1) = 10$ e $Q(s_7, a_2) = 5$

SARSA — tupla $(s_6, a_1, 0, s_7, a_2, \dots)$:

$$Q(s_6, a_1) \leftarrow 0 + \alpha(0 + \gamma Q(s_7, a_2)) = 2,5$$

Usa a_2 : foi a ação de fato tomada em s_7 , possivelmente por exploração.

Q-learning — tupla $(s_6, a_1, 0, s_7)$:

$$Q(s_6, a_1) \leftarrow 0 + \alpha(0 + \gamma \max_{a'} Q(s_7, a')) = 5$$

Usa $\max\{10, 5\} = 10$, ignorando a ação que o agente de fato escolheu.

Atenção

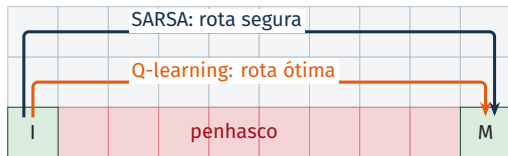
A inicialização de Q importa? **Assintoticamente não**, sob as condições do teorema anterior — mas **muito** no início. Inicializar otimisticamente é, aliás, uma estratégia de exploração.

Intuição

A diferença é de um símbolo, mas é ela que separa “qual o valor de seguir esta política, com exploração e tudo” de “qual o valor de jogar perfeitamente a partir daqui”.

O que essa diferença provoca: o *cliff walking*

Sutton & Barto (2018), Exemplo 6.6 — o exemplo canônico da distinção



$r = -1$ por passo; cair no penhasco custa -100 e devolve o agente a I. Se $\epsilon \rightarrow 0$, ambos convergem para a política ótima — mas *durante* o aprendizado o comportamento cauteloso do SARSA costuma ser preferível.

- ▶ **Q-learning** aprende o caminho *ótimo*, rente ao penhasco — mas, agindo com ϵ -guloso, cai nele de vez em quando e obtém **recompensa média pior**.
- ▶ **SARSA** aprende o caminho *seguro*, pela rota de cima, porque seu alvo já contabiliza os passos exploratórios.

Teste sua compreensão: SARSA e Q-learning

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Tanto SARSA quanto Q-learning podem atualizar a política após cada passo.
2. Se $\epsilon = 0$ em todos os instantes e Q é inicializado aleatoriamente, a atualização de SARSA coincide com a de Q-learning.

Pergunta para discussão conduzida em aula.

Teste sua compreensão: SARSA e Q-learning

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Tanto SARSA quanto Q-learning podem atualizar a política após cada passo.
2. Se $\epsilon = 0$ em todos os instantes e Q é inicializado aleatoriamente, a atualização de SARSA coincide com a de Q-learning.

Pergunta para discussão conduzida em aula.

Resposta

As duas são verdadeiras. (1) Ambos são incrementais e a política gulosa muda assim que Q muda. (2) Com $\epsilon = 0$ a política é gulosa pura, logo $a_{t+1} = \arg \max_{a'} Q(s_{t+1}, a')$ e os dois alvos coincidem. O que se perde é a garantia de convergência, que exige GLIE.

Um viés escondido no $\max$

Expansão: por que o Q-learning superestima valores

- ▶ O alvo do Q-learning usa $\max_{a'} \hat{Q}(s', a')$ como estimativa de $\max_{a'} Q(s', a')$. Mas o máximo de estimativas ruidosas é um estimador **enviesado para cima**:

$$\mathbb{E} \left[\max_{a'} \hat{Q}(s', a') \right] \geq \max_{a'} \mathbb{E} [\hat{Q}(s', a')]$$

pela desigualdade de Jensen ($\max$ é convexo).

- ▶ Basta que uma ação tenha tido sorte nas amostras para que o $\max$ a selecione — e o erro é propagado adiante pelo *bootstrapping*.

Intuição

A correção clássica é **desacoplar seleção e avaliação** da ação: usar um conjunto de estimativas para escolher $\arg \max$ e outro para avaliá-lo. É exatamente a ideia do *Double Q-learning* — e, depois, do *Double DQN*, no fim da aula.

SEÇÃO 4

Aproximação de função valor

04

Por que a tabela não basta

O momento em que $|S|$ deixa de ser um número que se escreve

Domínio	$ S $ aproximado
Mars Rover (Aula 2)	7
Gamão	10^{20}
Go (19×19)	10^{170}
Atari (4 quadros 84×84)	256^{28224}
Robô contínuo	incontável

Atenção

Não é só memória: mesmo que a tabela coubesse, seria preciso *visitar* cada entrada ao menos uma vez para estimá-la.

Intuição

A palavra-chave é **generalização**: aprender sobre um estado e, com isso, saber algo sobre estados parecidos que nunca foram visitados.

Queremos evitar armazenar por estado o modelo (P, R) , o valor V , o valor de ação Q e a própria política π — reduzindo **memória**, **computação** e, sobretudo, a **experiência** necessária.

O problema, primeiro com um oráculo

Reduzir RL a aprendizado supervisionado antes de complicar

- ▶ Suponha, por ora, que exista um oráculo capaz de devolver o valor verdadeiro $Q^\pi(s, a)$ para qualquer par consultado.
- ▶ Então temos um problema de regressão comum, sobre pares $((s, a), Q^\pi(s, a))$.
- ▶ Objetivo: achar $\mathbf{w}$ na família $\hat{Q}(s, a; \mathbf{w})$ que melhor aproxima Q^π .

Definição — Função de perda

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w}))^2 \right]$$

Como $\nabla_{\mathbf{w}} J(\mathbf{w}) = -2 \mathbb{E}_\pi [(Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})]$, o passo $\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$ por amostra fica

$$\Delta \mathbf{w} = \alpha (Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}).$$

SGD: usar uma (ou poucas) amostras no lugar da esperança. Em média, o passo é o mesmo do gradiente completo.

Aproximadores lineares e o caso tabular

Expansão: por que o tabular é um caso particular, e não uma alternativa

- ▶ Descreva cada par por um vetor de atributos $\mathbf{x}(s, a) = [x_1(s, a), \dots, x_d(s, a)]^\top$ e tome

$$\hat{Q}(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a), \quad \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}) = \mathbf{x}(s, a).$$

- ▶ A atualização vira, de forma particularmente limpa,

$$\Delta \mathbf{w} = \alpha (\text{alvo} - \mathbf{w}^\top \mathbf{x}(s, a)) \mathbf{x}(s, a).$$

Definição — O tabular como caso especial

Tome $d = |\mathcal{S}||\mathcal{A}|$ e atributos *one-hot*: $x_i(s, a) = \mathbf{1}\{(s, a) = i\}$. Então $\mathbf{w}$ é a tabela, $\nabla_{\mathbf{w}} \hat{Q}$ seleciona uma única coordenada, e a atualização acima reduz-se exatamente ao Q-learning tabular. Não há dois algoritmos: há um só, com representações diferentes — no caso one-hot simplesmente não existe generalização.

Sem oráculo: Monte Carlo com aproximação de função

O retorno G_t é amostra **não-enviesada**, ainda que ruidosa, de $Q^\pi(s_t, a_t)$: basta substituir o oráculo por G_t e fazer regressão sobre os pares $\langle (s_t, a_t), G_t \rangle$. Com aproximação linear, isso converge ao mínimo global da perda quadrática.

Algoritmo — Avaliação de política MC com aproximação de função

- 1: Inicialize $\mathbf{w}$; $k = 1$
- 2: **repita**
- 3: gere o episódio k seguindo π
- 4: **para** $t = 1, \dots, L_k$ **faça**
- 5: **se** primeira visita a (s_t, a_t) **então**
- 6: $G_t = \sum_{j=t}^{L_k} \gamma^{j-t} r_{k,j}$; $\mathbf{w} += \alpha (G_t - \hat{Q}(s_t, a_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$
- 7: **fim**
- 8: **fim**; $k += 1$
- 9: **fim**

TD(0) com aproximação de função

Três aproximações empilhadas, cada uma com seu preço

- ▶ No caso tabular, TD(0) atualizava $V^\pi(s) \leftarrow V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$, com o alvo $r + \gamma V^\pi(s')$ já *enviesado*.
- ▶ Com aproximação, o alvo passa a ser $r + \gamma \hat{V}^\pi(s'; \mathbf{w})$: **enviesado e** aproximado.

Atenção

1. **Amostragem** — usamos uma transição no lugar da esperança sobre P .
 2. **Bootstrapping** — usamos nossa própria estimativa no lugar do valor verdadeiro.
 3. **Aproximação de função** — projetamos o resultado numa família paramétrica que talvez não contenha V^π .
- ▶ A perda otimizada é $J(\mathbf{w}) = \mathbb{E}_\pi [(r_j + \gamma \hat{V}(s_{j+1}; \mathbf{w}) - \hat{V}(s_j; \mathbf{w}))^2]$.
 - ▶ **Detalhe importante:** tratamos o alvo como *constante* ao derivar. Por isso a atualização não é o gradiente verdadeiro de J — chama-se **semigradiente**.

TD(0) com aproximação: o algoritmo

Algoritmo — Avaliação de política TD(0) com aproximação de função

- 1: Inicialize $\mathbf{w}$ e o estado s
- 2: **repita**
- 3: amostre $a \sim \pi(s)$; observe $r(s, a)$ e $s' \sim P(\cdot \mid s, a)$
- 4: $\Delta \mathbf{w} = \alpha(r + \gamma \hat{V}(s'; \mathbf{w}) - \hat{V}(s; \mathbf{w})) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w})$; $\mathbf{w} += \Delta \mathbf{w}$
- 5: **se** s' não é terminal **então** $s \leftarrow s'$
- 6: **senão** reinicie o episódio, amostrando novo estado inicial s
- 7: **fim**

Intuição

Derivar também o alvo produziria o *gradiente residual*: estável, porém lento e exigindo duas amostras independentes do próximo estado. O semigradiente aprende mais rápido — ao custo das garantias.

SEÇÃO 5

Controle com aproximação e Deep Q-Learning

05

Controle com aproximação de função valor

- ▶ Representamos os valores de ação por $\hat{Q}^\pi(s, a; \mathbf{w}) \approx Q^\pi$.
- ▶ Intercalamos, como sempre:
 - avaliação aproximada, ajustando $\mathbf{w}$;
 - melhoria ϵ -gulosa sobre $\hat{Q}$.
- ▶ Como não conhecemos $Q^\pi(s_t, a_t)$, substituímos por um **alvo**, exatamente como no caso tabular:

Molde geral $\Delta \mathbf{w} = \alpha (\text{alvo} - \hat{Q}(s_t, a_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$

Monte Carlo alvo = G_t

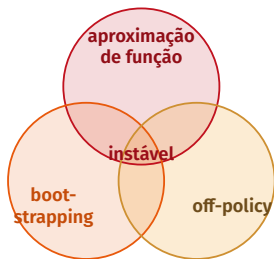
SARSA alvo = $r + \gamma \hat{Q}(s', a'; \mathbf{w})$

Q-learning alvo = $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w})$

Uma única linha de código, três algoritmos: toda a Seção 3 reaparece trocando “entrada da tabela” por “saída da rede”.

A “tríade mortal”

Quando os três se encontram, a convergência deixa de ser garantida



- ▶ Cada atualização faz um backup de Bellman aproximado e *projeta* o resultado sobre a família de funções.
- ▶ O backup é uma **contração**; a projeção pode ser uma **expansão**. A composição pode não contrair — e aí a sequência oscila ou diverge.
- ▶ Dois a dois são seguros; os três juntos, não.

Atenção

Contraexemplo canônico: o de **Baird** (Sutton & Barto, 2018), em que Q-learning linear diverge com os pesos crescendo sem limite.

Mapa das garantias de convergência

Expansão: o que se sabe, para avaliação e para controle

	Algoritmo	Tabular	Linear	Não-linear
Avaliação	MC	converge	converge	converge
	TD(0) on-policy	converge	converge [†]	pode divergir
	TD(0) off-policy	converge	pode divergir	pode divergir
Controle	MC-controle	converge	oscila próximo	pode divergir
	SARSA	converge	oscila próximo	pode divergir
	Q-learning	converge	pode divergir	pode divergir

[†] para um ponto fixo cujo erro é limitado por $\frac{1}{1-\gamma}$ vezes o erro da melhor aproximação da família.

Intuição

Note onde está o Q-learning com rede neural: sem garantia alguma. O DQN não resolve isso teoricamente — ele *engenheira* o problema para que, na prática, funcione.

Q-learning com redes neurais: o que dá errado

No caso tabular o Q-learning converge para Q^* ; com aproximação, ele minimiza um EQM por SGD usando um alvo estimado — e pode divergir.

Atenção

Dois problemas concretos, além da tríade:

1. **Correlação entre amostras.** Transições consecutivas são dependentes; SGD supõe amostras aproximadamente i.i.d.
2. **Alvos não-estacionários.** O alvo usa os mesmos pesos que estamos atualizando: perseguimos um alvo que se move.

Definição — As duas respostas do DQN (Mnih et al., 2015)

Repetição de experiência (*experience replay*) contra a correlação; **alvos fixos** (*fixed Q-targets*) contra a não-estacionariedade.

Repetição de experiência

Guardar o passado e reamostrá-lo fora de ordem

- ▶ Mantenha um **buffer de repetição** $\mathcal{D}$ com as tuplas (s, a, r, s') já vividas (tipicamente 10^6 transições, em janela deslizando).
- ▶ Repita:
 1. amostre $(s, a, r, s') \sim \mathcal{D}$ (ou um *minibatch*);
 2. calcule o alvo $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w})$;
 3. atualize $\Delta \mathbf{w} = \alpha (r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}) - \hat{Q}(s, a; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})$.

Intuição

Só é possível porque o Q-learning é **off-policy**: as tuplas do buffer foram geradas por políticas antigas, e isso não invalida o alvo (que já usa o $\max$, não a ação realmente escolhida). SARSA não pode fazer o mesmo sem correção. Resta, porém, o segundo problema: o alvo é tratado como escalar, mas muda quando os pesos mudam.

Alvos fixos

Congelar a rede que produz o alvo por C atualizações

- ▶ Mantenha **dois** conjuntos de pesos:
 - $\mathbf{w}$ — os pesos que estão sendo atualizados;
 - $\mathbf{w}^-$ — os pesos da *rede-alvo*, uma cópia congelada.
- ▶ O alvo passa a ser calculado com $\mathbf{w}^-$:

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-) - \hat{Q}(s, a; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})$$

- ▶ A cada C passos, sincronize: $\mathbf{w}^- \leftarrow \mathbf{w}$.

Intuição

Entre duas sincronizações, o problema vira uma regressão supervisionada comum, com rótulos fixos. É a manobra que devolve estabilidade a um procedimento que, de outro modo, persegue a própria cauda.

DQN: o algoritmo completo

Algoritmo — Deep Q-Network (Mnih et al., 2015)

- 1: Entrada: C , α , buffer $\mathcal{D} = \{\}$; inicialize $\mathbf{w}$, $\mathbf{w}^- \leftarrow \mathbf{w}$, $t = 0$; obtenha s_0
- 2: **repita**
- 3: amostre a_t pela política ϵ -gulosa sobre $\hat{Q}(s_t, \cdot; \mathbf{w})$
- 4: observe r_t e s_{t+1} ; armazene (s_t, a_t, r_t, s_{t+1}) em $\mathcal{D}$
- 5: amostre um *minibatch* aleatório $\{(s_i, a_i, r_i, s_{i+1})\}$ de $\mathcal{D}$
- 6: **para** cada i no *minibatch* **faça**
- 7: $y_i = r_i$ se o episódio terminou em $i + 1$; $y_i = r_i + \gamma \max_{a'} \hat{Q}(s_{i+1}, a'; \mathbf{w}^-)$ caso contrário
- 8: passo de descida em $(y_i - \hat{Q}(s_i, a_i; \mathbf{w}))^2$: $\Delta \mathbf{w} = \alpha (y_i - \hat{Q}(s_i, a_i; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_i, a_i; \mathbf{w})$
- 9: **fim**; $t += 1$
- 10: **se** $t \bmod C = 0$ **então** $\mathbf{w}^- \leftarrow \mathbf{w}$
- 11: **fim**

Há várias escolhas de projeto embutidas: arquitetura da rede, taxa de aprendizado, frequência C de sincronização, tamanho do buffer e como preenchê-lo antes de começar a aprender.

Teste sua compreensão: alvos fixos

Teste sua compreensão

No DQN o alvo usa um conjunto separado de pesos, $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$. Quais afirmações são verdadeiras?

1. Isso *dobra o tempo de computação* em relação a um método sem rede-alvo separada.
2. Isso *dobra o requisito de memória* em relação a um método sem rede-alvo separada.

Pergunta para discussão conduzida em aula.

Teste sua compreensão: alvos fixos

Teste sua compreensão

No DQN o alvo usa um conjunto separado de pesos, $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$. Quais afirmações são verdadeiras?

1. Isso *dobra o tempo de computação* em relação a um método sem rede-alvo separada.
2. Isso *dobra o requisito de memória* em relação a um método sem rede-alvo separada.

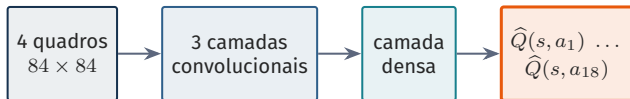
Pergunta para discussão conduzida em aula.

Resposta

Apenas a segunda. A memória dobra: são dois conjuntos de pesos armazenados. O tempo não dobra — em ambos os casos há uma passagem para frente (alvo) e uma para frente/para trás (atualização). O extra é só a cópia $\mathbf{w}^- \leftarrow \mathbf{w}$ a cada C passos.

DQN em Atari

Aprendizado ponta a ponta, de pixels a valores de ação



- ▶ Estado s : pilha dos **4 últimos quadros** de pixels brutos — para recuperar velocidade e direção, que um quadro único não revela.
- ▶ Saída: um valor $\hat{Q}(s, a)$ para cada uma das 18 posições de joystick, em uma única passagem.
- ▶ Recompensa: variação de pontuação (recortada em $[-1, 1]$).
- ▶ **A mesma arquitetura e os mesmos hiperparâmetros** em todos os jogos.

Quais ingredientes do DQN realmente importavam?

Estudo de ablação de Mnih et al. (2015): pontuação por jogo

Jogo	Linear	Rede profunda	DQN alvo fixo	DQN <i>replay</i>	DQN <i>replay</i> + alvo
Breakout	3	3	10	241	317
Enduro	62	29	141	831	1006
River Raid	2345	1453	2868	4102	7447
Seaquest	656	275	1003	823	2894
Space Invaders	301	302	373	826	1089

Atenção

Trocar o modelo linear por uma rede profunda, **sozinho**, chegou a *piorar* o desempenho (Enduro: 62 → 29; Seaquest: 656 → 275). O que fez a diferença foram o *replay* e o alvo fixo.

Por que o *replay* ajuda tanto

Três efeitos, não um

1. **Descorrelaciona** as amostras dentro do *minibatch*, aproximando a hipótese i.i.d. que o SGD supõe.
2. **Reaproveita** cada transição muitas vezes — ganho enorme de eficiência de dados, que é o recurso caro quando cada amostra exige uma interação com o ambiente.
3. **Estabiliza a distribuição de treino**: sem buffer, uma mudança de política altera imediatamente a distribuição dos dados, e a rede “esquece” o que aprendeu em regiões que deixou de visitar (esquecimento catastrófico).

Intuição

O terceiro efeito é o mais subestimado: o buffer age como uma memória que impede o agente de otimizar apenas para o presente. É por isso que o *replay* sozinho já produz saltos de $3 \rightarrow 241$ em Breakout.

Aperfeiçoamentos imediatos do DQN

- ▶ **Double DQN** (van Hasselt et al., AAAI 2016) — corrige o viés de maximização: a rede em treinamento *escolhe* a ação, a rede-alvo a *avalia*:

$$y = r + \gamma \hat{Q}(s', \arg \max_{a'} \hat{Q}(s', a'; \mathbf{w}); \mathbf{w}^-)$$

- ▶ **Repetição priorizada** (Schaul et al., ICLR 2016) — amostrar do buffer com probabilidade proporcional ao erro TD, em vez de uniformemente: aprender mais onde ainda se erra mais.
- ▶ **Dueling DQN** (Wang et al., ICML 2016) — decompor $\hat{Q}(s, a) = \hat{V}(s) + \hat{A}(s, a)$ em valor de estado e vantagem, aprendendo $\hat{V}(s)$ mesmo quando a ação é irrelevante.

Intuição

Os três são ortogonais e combináveis — é o que faz o *Rainbow* (Hessel et al., 2018), somando seis dessas extensões.

O que você deve levar desta aula

Definição — Capacidades esperadas

- ▶ Implementar avaliação de política por MC e TD(0), tabular e com aproximação de função.
- ▶ Implementar controle por MC, SARSA e Q-learning, e explicar a diferença entre os alvos de SARSA e de Q-learning.
- ▶ Enunciar as condições GLIE e Robbins–Monro e dizer para que cada uma serve.
- ▶ Listar os três elementos da tríade mortal — aproximação de função, *bootstrapping* e aprendizado off-policy — e descrever qualitativamente o problema que geram.
- ▶ Explicar as duas ideias que tornaram o DQN viável: repetição de experiência e alvos fixos.

Próxima aula

- ▶ Até aqui, todo método aprendeu uma **função valor** e derivou a política dela por $\arg \max$.
- ▶ Isso tem limites: ações contínuas tornam o $\arg \max$ caro, e políticas estocásticas ótimas não são representáveis dessa forma.
- ▶ **Próxima aula: gradientes de política** — parametrizar π_θ diretamente e subir o gradiente do retorno esperado.



Intuição

Guarde a tríade mortal: ela reaparece, com outra roupa, em quase todo algoritmo profundo do restante do curso.